

图论算法专题

算法学习笔记

2026 年 5 月 8 日

目录

1	图的存储与基础遍历	3
1.1	邻接矩阵存图	3
1.2	邻接表存图	4
1.3	深度优先搜索 (DFS)	5
1.4	广度优先搜索 (BFS)	7
2	最短路算法	9
2.1	Floyd-Warshall 算法	9
2.2	Dijkstra 算法 (朴素版)	10
2.3	Dijkstra 算法 (堆优化)	12
2.4	Bellman-Ford 算法	13
2.5	SPFA 算法	15
3	最小生成树	18
3.1	Kruskal 算法	18
3.2	Prim 算法	20
4	拓扑排序	22
5	强连通分量与缩点	24
5.1	Tarjan 求强连通分量	24
6	二分图匹配	27
6.1	匈牙利算法	27
6.2	Hopcroft-Karp 算法	29

7 网络流	31
7.1 Dinic 最大流算法	31
7.2 最小费用最大流 (SPFA 版本)	33
8 最近公共祖先 (LCA)	36
8.1 倍增法求 LCA	36
9 欧拉回路	39
9.1 Hierholzer 算法	39
10 Tarjan 桥与割点	41
10.1 求无向图的桥	41
10.2 求无向图的割点	43
11 差分约束系统	45
12 图论算法总结	49

1 图的存储与基础遍历

1.1 邻接矩阵存图

解决问题：适用于稠密图（边数接近 n^2 ），需要快速判断两点间是否有边、查询边权。
空间复杂度 $O(n^2)$ ，不适合稀疏图。

题目描述

给定 n 个顶点和 m 条有向边，每条边有边权 w 。使用邻接矩阵存储图，并输出每个顶点的所有出边。

输入示例

```
4 5
1 2 3
1 3 5
2 3 2
2 4 6
3 4 1
```

输出示例

```
顶点1的出边：1->2(3) 1->3(5)
顶点2的出边：2->3(2) 2->4(6)
顶点3的出边：3->4(1)
顶点4的出边：
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const int INF = 0x3f3f3f3f; // 无穷大常量，表示两点之间无边
4      int main() {
5          int n, m; // n: 顶点数量, m: 边的数量
6          cin >> n >> m;
7          vector<vector<int>>> g(n + 1, vector<int>(n + 1, INF)); // g[i][j]:
           从i到j的边权, INF表示无边
8          for (int i = 1; i <= n; i++) g[i][i] = 0; // 自己到自己的距离为0
9          for (int i = 0; i < m; i++) {
10             int u, v, w; // u: 起点, v: 终点, w: 边权
```

```
11         cin >> u >> v >> w;
12         g[u][v] = min(g[u][v], w); // 处理重边, 保留最小权值
13     }
14     // 输出每个顶点的出边
15     for (int i = 1; i <= n; i++) {
16         cout << "顶点" << i << "的出边: ";
17         for (int j = 1; j <= n; j++) {
18             if (g[i][j] != INF && i != j) {
19                 cout << i << "->" << j << "(" << g[i][j] << ") ";
20             }
21         }
22         cout << "\n";
23     }
24     return 0;
25 }
```

1.2 邻接表存图

解决问题: 适用于稀疏图 (边数远小于 n^2), 空间复杂度 $O(n + m)$, 配合 vector 实现。是竞赛中最常用的存图方式。

题目描述

给定 n 个顶点和 m 条有向边, 使用邻接表存储图, 并输出每个顶点的所有出边。

输入示例

```
4 5
1 2 3
1 3 5
2 3 2
2 4 6
3 4 1
```

输出示例

```
顶点1的出边: 1->2(3) 1->3(5)
顶点2的出边: 2->3(2) 2->4(6)
顶点3的出边: 3->4(1)
```

顶点4的出边:

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Edge {
4          int to, w; // to: 目标顶点, w: 边权
5      };
6      int main() {
7          int n, m; // n: 顶点数量, m: 边的数量
8          cin >> n >> m;
9          vector<vector<Edge>> adj(n + 1); // adj[i]: 从顶点i出发的所有边的列表
10         for (int i = 0; i < m; i++) {
11             int u, v, w; // u: 起点, v: 终点, w: 边权
12             cin >> u >> v >> w;
13             adj[u].push_back({v, w}); // 添加有向边 u->v
14         }
15         // 输出每个顶点的出边
16         for (int i = 1; i <= n; i++) {
17             cout << "顶点" << i << "的出边: ";
18             for (auto &e : adj[i]) {
19                 cout << i << "->" << e.to << "(" << e.w << ") ";
20             }
21             cout << "\n";
22         }
23         return 0;
24     }
```

1.3 深度优先搜索（DFS）

解决问题：用于遍历图的所有顶点，常用来判断图的连通性、寻找路径、拓扑排序（在有向无环图中进行 DFS 后序）、检测环（无向图通过记录父节点，有向图通过三色标记法）、求连通分量等。

题目描述

给定一个无向连通图，从顶点 1 开始进行深度优先搜索，输出遍历顺序。

输入示例

5 4

1 2

1 3

2 4

3 5

输出示例

1 2 4 3 5

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      vector<vector<int>> adj; // adj[i]: 顶点i的邻接表
4      vector<bool> vis; // vis[i]: 顶点i是否已被访问
5      // DFS递归函数: 访问顶点u
6      void dfs(int u) {
7          vis[u] = true;
8          cout << u << " "; // 输出访问顺序
9          for (int v : adj[u]) { // 遍历u的所有邻居v
10             if (!vis[v]) {
11                 dfs(v); // 递归访问未访问的邻居
12             }
13         }
14     }
15     int main() {
16         int n, m; // n: 顶点数, m: 边数
17         cin >> n >> m;
18         adj.resize(n + 1);
19         vis.assign(n + 1, false);
20         for (int i = 0; i < m; i++) {
21             int u, v; // u, v: 无向边的两端
22             cin >> u >> v;
23             adj[u].push_back(v);
24             adj[v].push_back(u); // 无向图添加反向边
25         }
26         dfs(1); // 从顶点1开始DFS
27         cout << "\n";
28         return 0;
29     }
```

1.4 广度优先搜索（BFS）

解决问题：求无权图的最短路径（边权为 1）、层次遍历、判断二分图（染色法配合 BFS）、求连通分量。BFS 按层次访问，先访问距离起点近的顶点。

题目描述

给定一个无向连通图，从顶点 1 开始进行广度优先搜索，输出遍历顺序。

输入示例

```
5 4
1 2
1 3
2 4
3 5
```

输出示例

```
1 2 3 4 5
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n, m; // n: 顶点数, m: 边数
5          cin >> n >> m;
6          vector<vector<int>> adj(n + 1); // adj[i]: 顶点i的邻接表
7          for (int i = 0; i < m; i++) {
8              int u, v; // u, v: 无向边的两端
9              cin >> u >> v;
10             adj[u].push_back(v);
11             adj[v].push_back(u);
12         }
13         vector<bool> vis(n + 1, false); // vis[i]: 顶点i是否已入队
14         queue<int> q; // q: BFS队列
15         q.push(1); // 起点入队
16         vis[1] = true;
17         while (!q.empty()) {
18             int u = q.front(); // 取出队首顶点
19             q.pop();
```

```
20         cout << u << " "; // 输出访问顺序
21     for (int v : adj[u]) { // 遍历u的所有邻居
22         if (!vis[v]) {
23             vis[v] = true; // 标记为已访问（入队时即标记，防止重复入队）
24             q.push(v);
25         }
26     }
27 }
28 cout << "\n";
29 return 0;
30 }
```


2 最短路算法

2.1 Floyd-Warshall 算法

解决问题：求图中任意两点间的最短距离，可处理负权边（但不能有负环）。时间复杂度 $O(n^3)$ ，适用于 $n \leq 500$ 的稠密图，或需要多次查询任意两点最短路的情况。核心思想为动态规划，枚举中间点进行松弛。

题目描述

给定一个带权有向图，求任意两点间的最短距离。若两点不可达输出 `INF`。

输入示例

```
4 5
1 2 3
1 3 5
2 3 2
2 4 6
3 4 1
```

输出示例

最短距离矩阵：

```
0 3 5 6
INF 0 2 3
INF INF 0 1
INF INF INF 0
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const long long INF = 1e18; // 极大值，表示不可达
4      int main() {
5          int n, m; // n: 顶点数, m: 边数
6          cin >> n >> m;
7          vector<vector<long long>>> dist(n + 1, vector<long long>(n + 1, INF))
            ; // dist[i][j]: i到j的最短距离
8          for (int i = 1; i <= n; i++) dist[i][i] = 0; // 自己到自己的距离为0
9          for (int i = 0; i < m; i++) {
```

```

10     int u, v, w; // u: 起点, v: 终点, w: 边权
11     cin >> u >> v >> w;
12     dist[u][v] = min(dist[u][v], (long long)w); // 处理重边, 取最小值
13 }
14 // Floyd核心: 三重循环, 枚举中间点k, 尝试松弛i->j的路径
15 for (int k = 1; k <= n; k++) {
16     for (int i = 1; i <= n; i++) {
17         for (int j = 1; j <= n; j++) {
18             if (dist[i][k] != INF && dist[k][j] != INF) {
19                 // 如果i->k和k->j都可达, 尝试更新i->j
20                 dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
21             }
22         }
23     }
24 }
25 cout << "最短距离矩阵:\n";
26 for (int i = 1; i <= n; i++) {
27     for (int j = 1; j <= n; j++) {
28         if (dist[i][j] == INF) cout << "INF ";
29         else cout << dist[i][j] << " ";
30     }
31     cout << "\n";
32 }
33 return 0;
34 }

```

2.2 Dijkstra 算法（朴素版）

解决问题：求非负权图（所有边权 ≥ 0 ）中单源最短路，即从一个起点到其他所有顶点的最短距离。时间复杂度 $O(n^2)$ ，适合稠密图（ $n \leq 10^4, m \approx n^2$ ）。核心思想：每次从未确定最短距离的顶点中选择距离最小的进行松弛。

题目描述

给定一个带权有向图（边权非负），求从顶点 1 到其他所有顶点的最短距离。

输入示例

```

4 5
1 2 2

```

```

1 3 5
2 3 1
2 4 7
3 4 3

```

输出示例

顶点1到各顶点的最短距离：

```

到1: 0
到2: 2
到3: 3
到4: 6

```

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      const int INF = 0x3f3f3f3f; // 无穷大
4      int main() {
5          int n, m; // n: 顶点数, m: 边数
6          cin >> n >> m;
7          vector<vector<pair<int, int>>> adj(n + 1); // adj[u] 存储 (v, w) 表示
            u->v 权值为w
8          for (int i = 0; i < m; i++) {
9              int u, v, w; // u: 起点, v: 终点, w: 边权
10             cin >> u >> v >> w;
11             adj[u].push_back({v, w});
12         }
13         vector<int> dist(n + 1, INF); // dist[i]: 起点到i的最短距离
14         vector<bool> vis(n + 1, false); // vis[i]: 顶点i的最短距离是否已确定
15         dist[1] = 0; // 起点到自身距离为0
16         // Dijkstra核心: 每次从未确定的点中选距离最小的
17         for (int i = 1; i <= n; i++) {
18             int u = -1; // u: 本轮选出的未确定顶点中dist最小的
19             for (int j = 1; j <= n; j++) {
20                 if (!vis[j] && (u == -1 || dist[j] < dist[u])) {
21                     u = j;
22                 }
23             }
24             if (u == -1 || dist[u] == INF) break; // 剩余顶点不可达

```

```

25     vis[u] = true; // 确定u的最短距离
26     for (auto &p : adj[u]) { // 用u松弛其邻居
27         int v = p.first, w = p.second;
28         if (dist[u] + w < dist[v]) {
29             dist[v] = dist[u] + w; // 松弛操作
30         }
31     }
32 }
33 cout << "顶点1到各顶点的最短距离:\n";
34 for (int i = 1; i <= n; i++) {
35     cout << "到" << i << ": ";
36     if (dist[i] == INF) cout << "不可达\n";
37     else cout << dist[i] << "\n";
38 }
39 return 0;
40 }

```

2.3 Dijkstra 算法（堆优化）

解决问题:非负权图的单源最短路,时间复杂度 $O(m \log n)$,适合稀疏图($n, m \leq 2 \times 10^5$)。使用优先队列（小根堆）优化”选择最小距离顶点”的过程。是竞赛中最常用的最短路算法之一。

题目描述

同朴素版 Dijkstra。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     const long long INF = 1e18; // 极大值
4     int main() {
5         int n, m; // n: 顶点数, m: 边数
6         cin >> n >> m;
7         vector<vector<pair<int, long long>>> adj(n + 1); // adj[u]: (v, w)
8         // 表示 u->v 权w
9         for (int i = 0; i < m; i++) {
10             int u, v; long long w; // u: 起点, v: 终点, w: 边权
11             cin >> u >> v >> w;
12             adj[u].push_back({v, w});
13         }
14         vector<long long> dist(n + 1, INF); // dist[i]: 起点到i的最短距离

```

```

14     dist[1] = 0;
15     // 小根堆，存储（距离，顶点），按距离升序排列
16     priority_queue<pair<long long, int>, vector<pair<long long, int>>,
17         greater<>> pq;
18     pq.push({0, 1}); // 起点入堆
19     while (!pq.empty()) {
20         auto [d, u] = pq.top(); // d: 当前记录的起点到u的距离
21         pq.pop();
22         if (d > dist[u]) continue; // 若堆顶记录的距离大于已确定的最短距
23             离，跳过（惰性删除）
24         for (auto &p : adj[u]) { // 松弛u的所有出边
25             int v = p.first;
26             long long w = p.second;
27             if (dist[u] + w < dist[v]) { // 若通过u到v更近
28                 dist[v] = dist[u] + w; // 更新距离
29                 pq.push({dist[v], v}); // 新距离入堆
30             }
31         }
32     }
33     cout << "顶点1到各顶点的最短距离:\n";
34     for (int i = 1; i <= n; i++) {
35         cout << "到" << i << ": ";
36         if (dist[i] == INF) cout << "不可达\n";
37         else cout << dist[i] << "\n";
38     }
39     return 0;
40 }

```

2.4 Bellman-Ford 算法

解决问题：求含负权边的单源最短路，并能检测负环。时间复杂度 $O(nm)$ ，适用于 $n, m \leq 5000$ 且边权可能为负的情况。核心思想：对所有边进行 $n - 1$ 轮松弛操作，若第 n 轮仍能松弛则说明存在负环。

题目描述

给定一个带权有向图（可能含负权边），求从顶点 1 到其他顶点的最短距离，并判断是否存在负环（从起点可达的负环）。

输入示例

```
4 4
1 2 3
2 3 -4
3 4 2
4 2 -1
```

输出示例

存在负环!

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const long long INF = 1e18; // 极大值
4      struct Edge {
5          int u, v; long long w; // u: 起点, v: 终点, w: 边权
6      };
7      int main() {
8          int n, m; // n: 顶点数, m: 边数
9          cin >> n >> m;
10         vector<Edge> edges(m); // edges: 存储所有边的数组
11         for (int i = 0; i < m; i++) {
12             cin >> edges[i].u >> edges[i].v >> edges[i].w;
13         }
14         vector<long long> dist(n + 1, INF); // dist[i]: 起点到i的最短距离
15         dist[1] = 0;
16         // Bellman-Ford核心: 进行n-1轮松弛
17         for (int i = 1; i <= n - 1; i++) {
18             bool updated = false; // 本轮是否有松弛发生
19             for (auto &e : edges) {
20                 if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) {
21                     dist[e.v] = dist[e.u] + e.w; // 松弛操作
22                     updated = true;
23                 }
24             }
25             if (!updated) break; // 若无松弛, 提前结束
26         }
27         // 第n轮松弛检测负环
```

```

28     bool has_negative_cycle = false;
29     for (auto &e : edges) {
30         if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) {
31             has_negative_cycle = true; // 仍能松弛, 说明存在负环
32             break;
33         }
34     }
35     if (has_negative_cycle) {
36         cout << "存在负环!\n";
37     } else {
38         cout << "顶点1到各顶点的最短距离:\n";
39         for (int i = 1; i <= n; i++) {
40             cout << "到" << i << ": ";
41             if (dist[i] == INF) cout << "不可达\n";
42             else cout << dist[i] << "\n";
43         }
44     }
45     return 0;
46 }

```

2.5 SPFA 算法

解决问题: Bellman-Ford 的队列优化版本, 平均时间复杂度 $O(km)$, 最坏 $O(nm)$ 。同样支持负权边, 可检测负环 (通过记录每个顶点入队次数, 若超过 n 次则有负环)。实际竞赛中可能被卡成 $O(nm)$, 需谨慎使用。

题目描述

同 Bellman-Ford, 使用 SPFA 求解。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     const long long INF = 1e18;
4     int main() {
5         int n, m; // n: 顶点数, m: 边数
6         cin >> n >> m;
7         vector<vector<pair<int, long long>>> adj(n + 1); // adj[u]: (v, w)
8         for (int i = 0; i < m; i++) {
9             int u, v; long long w; // u: 起点, v: 终点, w: 边权
10            cin >> u >> v >> w;
11            adj[u].push_back({v, w});

```

```

12     }
13     vector<long long> dist(n + 1, INF);
14     vector<bool> inq(n + 1, false); // inq[i]: 顶点i是否在队列中
15     vector<int> cnt(n + 1, 0); // cnt[i]: 顶点i入队次数, 用于检测负环
16     dist[1] = 0;
17     queue<int> q; // q: SPFA队列
18     q.push(1);
19     inq[1] = true;
20     cnt[1] = 1;
21     bool neg_cycle = false; // neg_cycle: 是否存在负环
22     while (!q.empty()) {
23         int u = q.front(); // 取出队首顶点
24         q.pop();
25         inq[u] = false;
26         for (auto &p : adj[u]) { // 松弛u的所有出边
27             int v = p.first;
28             long long w = p.second;
29             if (dist[u] + w < dist[v]) {
30                 dist[v] = dist[u] + w; // 松弛
31                 if (!inq[v]) { // 若v不在队列中, 入队
32                     q.push(v);
33                     inq[v] = true;
34                     cnt[v]++;
35                     if (cnt[v] > n) { // 入队超过n次, 存在负环
36                         neg_cycle = true;
37                         break;
38                     }
39                 }
40             }
41         }
42         if (neg_cycle) break;
43     }
44     if (neg_cycle) {
45         cout << "存在负环!\n";
46     } else {
47         cout << "顶点1到各顶点的最短距离:\n";
48         for (int i = 1; i <= n; i++) {
49             cout << "到" << i << ": ";
50             if (dist[i] == INF) cout << "不可达\n";
51             else cout << dist[i] << "\n";

```



```
52         }  
53     }  
54     return 0;  
55 }
```

3 最小生成树

3.1 Kruskal 算法

解决问题:求无向连通图的最小生成树(边权和最小的生成树)。时间复杂度 $O(m \log m)$, 适用于稀疏图。核心思想: 将所有边按权值升序排序, 使用并查集维护连通性, 依次选择不形成环的边加入生成树。

题目描述

给定一个无向连通图, 求最小生成树的边权和, 并输出选择的边。

输入示例

```
5 7
1 2 3
1 3 5
2 3 1
2 4 4
3 4 6
3 5 2
4 5 7
```

输出示例

最小生成树边权和: 10

选择的边:

```
2 - 3 : 1
3 - 5 : 2
1 - 2 : 3
2 - 4 : 4
```

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  struct Edge {
4      int u, v, w; // u, v: 边的两端, w: 边权
5      bool operator<(const Edge &other) const {
6          return w < other.w; // 按边权升序排序
7      }
```

```
8     };
9     vector<int> parent; // parent[i]: 并查集中顶点i的父节点
10    // 并查集查找操作 (路径压缩)
11    int find(int x) {
12        if (parent[x] != x) parent[x] = find(parent[x]); // 递归查找根并路径
13        // 压缩
14        return parent[x];
15    }
16    // 并查集合并操作
17    void unite(int x, int y) {
18        int rx = find(x), ry = find(y);
19        if (rx != ry) parent[rx] = ry; // 将一棵树挂到另一棵上
20    }
21    int main() {
22        int n, m; // n: 顶点数, m: 边数
23        cin >> n >> m;
24        vector<Edge> edges(m); // edges: 所有边的数组
25        for (int i = 0; i < m; i++) {
26            cin >> edges[i].u >> edges[i].v >> edges[i].w;
27        }
28        // 按边权升序排序
29        sort(edges.begin(), edges.end());
30        // 初始化并查集
31        parent.resize(n + 1);
32        for (int i = 1; i <= n; i++) parent[i] = i; // 每个顶点初始为独立的集
33        // 合
34        long long total = 0; // total: 最小生成树的总边权
35        vector<Edge> mst; // mst: 存储选入生成树的边
36        // Kruskal核心: 遍历排序后的边, 选择不形成环的边
37        for (auto &e : edges) {
38            if (find(e.u) != find(e.v)) { // 若u和v不在同一集合, 则加入边e不会
39                // 形成环
40                unite(e.u, e.v); // 合并集合
41                total += e.w;
42                mst.push_back(e);
43                if ((int)mst.size() == n - 1) break; // 已选n-1条边, 生成树构
44                // 建完毕
45            }
46        }
47        cout << "最小生成树边权和: " << total << "\n";
```

```

44     cout << "选择的边:\n";
45     for (auto &e : mst) {
46         cout << e.u << " - " << e.v << " : " << e.w << "\n";
47     }
48     return 0;
49 }

```

3.2 Prim 算法

解决问题：求无向连通图的最小生成树。时间复杂度朴素 $O(n^2)$ 适合稠密图，堆优化 $O(m \log n)$ 适合稀疏图。核心思想：从任意顶点开始，每次将距离生成树最近的顶点加入树中，与 Dijkstra 思路相似。

题目描述

同 Kruskal，使用 Prim 求解。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     const int INF = 0x3f3f3f3f;
4     int main() {
5         int n, m; // n: 顶点数, m: 边数
6         cin >> n >> m;
7         vector<vector<pair<int, int>>> adj(n + 1); // adj[u]: (v, w)
8         for (int i = 0; i < m; i++) {
9             int u, v, w; // u, v: 边的两端, w: 边权
10            cin >> u >> v >> w;
11            adj[u].push_back({v, w});
12            adj[v].push_back({u, w}); // 无向图添加双向边
13        }
14        vector<int> min_edge(n + 1, INF); // min_edge[i]: 顶点i到当前生成树的
            最小边权
15        vector<bool> in_mst(n + 1, false); // in_mst[i]: 顶点i是否已在生成树
            中
16        min_edge[1] = 0; // 从顶点1开始构建
17        long long total = 0; // total: 最小生成树总边权
18        // Prim核心: 每次选min_edge最小的未加入顶点
19        for (int i = 1; i <= n; i++) {
20            int u = -1; // u: 本轮选出的顶点
21            for (int j = 1; j <= n; j++) {
22                if (!in_mst[j] && (u == -1 || min_edge[j] < min_edge[u])) {

```

```
23         u = j;
24     }
25 }
26 if (min_edge[u] == INF) { // 图不连通
27     cout << "图不连通\n";
28     return 0;
29 }
30 in_mst[u] = true; // 将u加入生成树
31 total += min_edge[u];
32 for (auto &p : adj[u]) { // 用u的邻边更新其他顶点到生成树的距离
33     int v = p.first, w = p.second;
34     if (!in_mst[v] && w < min_edge[v]) {
35         min_edge[v] = w; // 更新v到生成树的最小边权
36     }
37 }
38 }
39 cout << "最小生成树边权和: " << total << "\n";
40 return 0;
41 }
```

4 拓扑排序

解决问题: 给定有向无环图 (DAG), 输出顶点的线性序列, 使得对于每条有向边 $u \rightarrow v$, u 在序列中出现在 v 之前。用于任务调度、依赖关系分析 (如课程先修关系)、编译器的文件依赖编译顺序等。若存在环则无解。

题目描述

给定一个有向图, 输出一个拓扑序。若存在环则输出”存在环”。

输入示例

```
6 6
1 2
1 3
2 4
3 4
4 5
6 3
```

输出示例

拓扑序: 1 6 2 3 4 5

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n, m; // n: 顶点数, m: 边数
5          cin >> n >> m;
6          vector<vector<int>> adj(n + 1); // adj[i]: 顶点i的出边列表
7          vector<int> indeg(n + 1, 0); // indeg[i]: 顶点i的入度
8          for (int i = 0; i < m; i++) {
9              int u, v; // u: 边的起点, v: 边的终点 (u->v)
10             cin >> u >> v;
11             adj[u].push_back(v);
12             indeg[v]++; // v的入度加1
13         }
14         queue<int> q; // q: 存储当前入度为0的顶点
15         for (int i = 1; i <= n; i++) {
```

```
16         if (indeg[i] == 0) q.push(i); // 入度为0的入队
17     }
18     vector<int> topo; // topo: 存储拓扑序
19     while (!q.empty()) {
20         int u = q.front(); // 取出入度为0的顶点
21         q.pop();
22         topo.push_back(u); // 加入拓扑序
23         for (int v : adj[u]) { // 遍历u的所有出边
24             indeg[v]--; // 删除边u->v, v的入度减1
25             if (indeg[v] == 0) { // 若v入度变为0
26                 q.push(v); // 入队
27             }
28         }
29     }
30     if ((int)topo.size() < n) { // 拓扑序不足n个顶点, 说明有环
31         cout << "存在环\n";
32     } else {
33         cout << "拓扑序: ";
34         for (int x : topo) cout << x << " ";
35         cout << "\n";
36     }
37     return 0;
38 }
```

5 强连通分量与缩点

5.1 Tarjan 求强连通分量

解决问题：求有向图的强连通分量（SCC），即分量内任意两点可以互达。时间复杂度 $O(n + m)$ 。可以用于缩点：将每个 SCC 缩为一个点，原图变成 DAG，从而能在上面进行拓扑排序、DP 等。常用来解决传递关系简化问题。

题目描述

给定一个有向图，求强连通分量的个数，并输出每个顶点所属的 SCC 编号。

输入示例

```
6 8
1 2
2 3
3 1
3 4
4 5
5 6
6 4
2 5
```

输出示例

```
强连通分量数量：3
顶点1属于SCC 2
顶点2属于SCC 2
顶点3属于SCC 2
顶点4属于SCC 1
顶点5属于SCC 1
顶点6属于SCC 1
```

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  vector<vector<int>> adj; // adj[i]: 顶点i的邻接表
4  vector<int> dfn, low; // dfn[i]: DFS序, low[i]: i能回溯到的最早dfn
5  vector<bool> in_stk; // in_stk[i]: 顶点i是否在栈中
```



```

6     stack<int> stk; // stk: Tarjan算法用的栈
7     int timer = 0; // timer: 当前DFS时间戳
8     int scc_cnt = 0; // scc_cnt: SCC计数器
9     vector<int> belong; // belong[i]: 顶点i所属SCC编号
10    // Tarjan核心: DFS并追溯low值
11    void tarjan(int u) {
12        dfn[u] = low[u] = ++timer; // 初始化dfn和low
13        stk.push(u);
14        in_stk[u] = true;
15        for (int v : adj[u]) {
16            if (!dfn[v]) { // 若v未被访问
17                tarjan(v);
18                low[u] = min(low[u], low[v]); // 用子节点的low更新当前节点
19            } else if (in_stk[v]) { // 若v在栈中 (属于当前SCC)
20                low[u] = min(low[u], dfn[v]); // 用v的dfn更新low
21            }
22        }
23        // 若dfn[u] == low[u], 则u是一个SCC的根
24        if (dfn[u] == low[u]) {
25            scc_cnt++;
26            int v;
27            do {
28                v = stk.top(); // 弹出栈顶
29                stk.pop();
30                in_stk[v] = false;
31                belong[v] = scc_cnt; // 标记v所属SCC
32            } while (v != u); // 直到弹出u为止
33        }
34    }
35    int main() {
36        int n, m; // n: 顶点数, m: 边数
37        cin >> n >> m;
38        adj.resize(n + 1);
39        dfn.assign(n + 1, 0);
40        low.resize(n + 1);
41        in_stk.assign(n + 1, false);
42        belong.resize(n + 1);
43        for (int i = 0; i < m; i++) {
44            int u, v; // u, v: 有向边 u->v
45            cin >> u >> v;

```

```
46         adj[u].push_back(v);
47     }
48     // 对每个未访问的顶点进行Tarjan
49     for (int i = 1; i <= n; i++) {
50         if (!dfn[i]) tarjan(i);
51     }
52     cout << "强连通分量数量: " << scc_cnt << "\n";
53     for (int i = 1; i <= n; i++) {
54         cout << "顶点 " << i << "属于SCC " << belong[i] << "\n";
55     }
56     return 0;
57 }
```

6 二分图匹配

6.1 匈牙利算法

解决问题：求二分图的最大匹配（边数最多的匹配，使得匹配边没有公共顶点）。时间复杂度 $O(n \times m)$ 。常用于任务分配问题，如工人分配任务、婚配问题、二分图覆盖问题等。

题目描述

给定一个二分图，左侧有 n 个顶点，右侧有 m 个顶点，有 e 条边连接左右两侧。求最大匹配数，并输出匹配方案。

输入示例

```
3 3 5
1 1
1 2
2 2
2 3
3 2
```

输出示例

```
最大匹配数：3
匹配方案：
左1 -> 右1
左2 -> 右3
左3 -> 右2
```

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  vector<vector<int>> adj; // adj[i]: 左侧顶点i能匹配的右侧顶点列表
4  vector<int> match; // match[v]: 右侧顶点v当前匹配的左侧顶点, 0表示未匹配
5  vector<bool> vis; // vis[v]: 本轮增广中右侧顶点v是否被访问过
6  // 匈牙利算法核心: 尝试为左侧顶点u寻找匹配
7  bool dfs(int u) {
8      for (int v : adj[u]) { // 遍历u能匹配的所有右侧顶点
9          if (!vis[v]) { // 若v在本轮未被访问
```

```
10         vis[v] = true;
11         // 若v未匹配, 或v的当前匹配者可以找到新的匹配
12         if (match[v] == 0 || dfs(match[v])) {
13             match[v] = u; // 将v匹配给u
14             return true; // 匹配成功
15         }
16     }
17 }
18 return false; // 匹配失败
19 }
20 int main() {
21     int n, m, e; // n: 左侧顶点数, m: 右侧顶点数, e: 边数
22     cin >> n >> m >> e;
23     adj.resize(n + 1);
24     match.assign(m + 1, 0);
25     for (int i = 0; i < e; i++) {
26         int u, v; // u: 左侧顶点, v: 右侧顶点 (左u 到 右v)
27         cin >> u >> v;
28         adj[u].push_back(v);
29     }
30     int ans = 0; // ans: 最大匹配数
31     // 对每个左侧顶点尝试增广
32     for (int i = 1; i <= n; i++) {
33         vis.assign(m + 1, false); // 每轮清空访问标记
34         if (dfs(i)) ans++; // 若为i找到增广路, 匹配数+1
35     }
36     cout << "最大匹配数: " << ans << "\n";
37     cout << "匹配方案:\n";
38     for (int v = 1; v <= m; v++) {
39         if (match[v] != 0) {
40             cout << "左" << match[v] << " -> 右" << v << "\n";
41         }
42     }
43     return 0;
44 }
```

6.2 Hopcroft-Karp 算法

解决问题：二分图最大匹配，时间复杂度 $O(\sqrt{n} \times m)$ ，比匈牙利算法快，适合大数据量。同时使用 BFS 分层和 DFS 增广。

题目描述

同匈牙利算法，使用 HK 算法。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      const int INF = 0x3f3f3f3f;
4      vector<vector<int>> adj; // adj[i]: 左侧顶点i的邻接表
5      vector<int> matchL, matchR; // matchL[i]: 左侧i匹配的右侧顶点, matchR[i]:
        右侧i匹配的左侧顶点
6      vector<int> dist; // dist[i]: 左侧顶点i在BFS分层中的距离
7      // BFS分层: 为所有未匹配的左侧顶点建立层次
8      bool bfs(int n) {
9          queue<int> q;
10         for (int i = 1; i <= n; i++) {
11             if (matchL[i] == 0) { // 未匹配的左侧顶点作为起点
12                 dist[i] = 0;
13                 q.push(i);
14             } else {
15                 dist[i] = INF;
16             }
17         }
18         dist[0] = INF; // 0作为哨兵
19         while (!q.empty()) {
20             int u = q.front(); q.pop();
21             if (dist[u] < dist[0]) {
22                 for (int v : adj[u]) {
23                     int mv = matchR[v]; // v当前匹配的左侧顶点
24                     if (dist[mv] == INF) { // 若该路径未访问过
25                         dist[mv] = dist[u] + 1;
26                         q.push(mv);
27                     }
28                 }
29             }
30         }
31         return dist[0] != INF; // 若存在增广路则返回true
32     }

```

```
33 // DFS增广：从左侧顶点u出发找增广路
34 bool dfs(int u) {
35     if (u != 0) {
36         for (int v : adj[u]) {
37             int mv = matchR[v];
38             if (dist[mv] == dist[u] + 1 && dfs(mv)) {
39                 matchR[v] = u;
40                 matchL[u] = v;
41                 return true;
42             }
43         }
44         dist[u] = INF;
45         return false;
46     }
47     return true; // 到达哨兵，增广成功
48 }
49 int main() {
50     int n, m, e; // n: 左侧点数, m: 右侧点数, e: 边数
51     cin >> n >> m >> e;
52     adj.resize(n + 1);
53     matchL.assign(n + 1, 0);
54     matchR.assign(m + 1, 0);
55     dist.resize(n + 1);
56     for (int i = 0; i < e; i++) {
57         int u, v; cin >> u >> v;
58         adj[u].push_back(v);
59     }
60     int ans = 0;
61     while (bfs(n)) { //BFS分层
62         for (int i = 1; i <= n; i++) {
63             if (matchL[i] == 0 && dfs(i)) ans++; // 对每个未匹配点进行DFS
64                 增广
65         }
66     }
67     cout << "最大匹配数: " << ans << "\n";
68     return 0;
69 }
```

7 网络流

7.1 Dinic 最大流算法

解决问题：求有向图（容量网络）中从源点 S 到汇点 T 的最大流量。每条边有容量限制。时间复杂度 $O(n^2m)$ ，实际很快。常用于流量分配问题、二分图最大匹配（转化成流网络）、最小割问题等。

题目描述

给定一个有向容量网络，求从源点 S 到汇点 T 的最大流。

输入示例

```
4 5 1 4
1 2 10
1 3 5
2 3 15
2 4 10
3 4 10
```

输出示例

最大流：15

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const long long INF = 1e18;
4      struct Edge {
5          int to; // to: 目标顶点
6          long long cap; // cap: 剩余容量
7          int rev; // rev: 反向边在邻接表中的下标
8      };
9      vector<vector<Edge>> graph; // graph[i]: 顶点i的邻接边列表
10     vector<int> level; // level[i]: 顶点i在BFS分层中的层次
11     vector<int> iter; // iter[i]: 顶点i当前弧优化指针
12     // BFS构建分层图
13     void bfs(int s) {
14         level.assign(graph.size(), -1);
```

```

15     queue<int> q;
16     level[s] = 0; // 源点层次为0
17     q.push(s);
18     while (!q.empty()) {
19         int u = q.front(); q.pop();
20         for (auto &e : graph[u]) {
21             if (e.cap > 0 && level[e.to] < 0) { // 有剩余容量且未分层
22                 level[e.to] = level[u] + 1;
23                 q.push(e.to);
24             }
25         }
26     }
27 }
28 // DFS寻找增广路
29 long long dfs(int u, int t, long long f) {
30     if (u == t) return f; // 到达汇点, 返回可增广流量
31     for (int &i = iter[u]; i < (int)graph[u].size(); i++) { // 当前弧优化
32         Edge &e = graph[u][i];
33         if (e.cap > 0 && level[u] < level[e.to]) { // 容量>0且层次递增
34             long long d = dfs(e.to, t, min(f, e.cap)); // 递归增广
35             if (d > 0) {
36                 e.cap -= d; // 正向边减容量
37                 graph[e.to][e.rev].cap += d; // 反向边加容量
38                 return d;
39             }
40         }
41     }
42     return 0; // 无法增广
43 }
44 // 添加有向边
45 void add_edge(int from, int to, long long cap) {
46     graph[from].push_back({to, cap, (int)graph[to].size()});
47     graph[to].push_back({from, 0, (int)graph[from].size() - 1}); // 反向
48                                     边初始容量为0
49 }
50 int main() {
51     int n, m, s, t; // n: 顶点数, m: 边数, s: 源点, t: 汇点
52     cin >> n >> m >> s >> t;
53     graph.resize(n + 1);
54     level.resize(n + 1);

```



```

54     iter.resize(n + 1);
55     for (int i = 0; i < m; i++) {
56         int u, v; long long w; // u: 起点, v: 终点, w: 容量
57         cin >> u >> v >> w;
58         add_edge(u, v, w);
59     }
60     long long max_flow = 0; // max_flow: 最大流
61     while (true) {
62         bfs(s); // 构建分层图
63         if (level[t] < 0) break; // 若汇点不可达, 结束
64         fill(iter.begin(), iter.end(), 0); // 重置当前弧
65         long long f;
66         while ((f = dfs(s, t, INF)) > 0) { // 不断增广直到无法找到增广路
67             max_flow += f;
68         }
69     }
70     cout << "最大流: " << max_flow << "\n";
71     return 0;
72 }

```

7.2 最小费用最大流 (SPFA 版本)

解决问题：在容量网络中，每条边还有单位流量费用。求从源点到汇点流量为 f 的最小费用，或流量不固定时求最大流的最小费用。时间复杂度 $O(F \times nm)$ ， F 为最大流量。常用于运输问题、分配问题等。

题目描述

给定一个容量网络，每条边有容量和单位费用。求从 S 到 T 流量最大时的最小总费用。

输入示例

```

4 5 1 4
1 2 10 2
1 3 5 3
2 3 15 1
2 4 10 3
3 4 10 2

```

输出示例

最大流: 15

最小费用: 55

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const int INF = 0x3f3f3f3f;
4      struct Edge {
5          int to, cap, cost, rev; // to: 目标, cap: 容量, cost: 单位费用, rev:
                                   // 反向边索引
6      };
7      vector<vector<Edge>> graph;
8      // 添加边
9      void add_edge(int from, int to, int cap, int cost) {
10         graph[from].push_back({to, cap, cost, (int)graph[to].size()});
11         graph[to].push_back({from, 0, -cost, (int)graph[from].size() - 1});
12         // 反向边费用为-cost
13     }
14     int main() {
15         int n, m, s, t; // n: 顶点数, m: 边数, s: 源, t: 汇
16         cin >> n >> m >> s >> t;
17         graph.resize(n + 1);
18         for (int i = 0; i < m; i++) {
19             int u, v, cap, cost; // u: 起点, v: 终点, cap: 容量, cost: 单位费
                                   // 用
20             cin >> u >> v >> cap >> cost;
21             add_edge(u, v, cap, cost);
22         }
23         int max_flow = 0, min_cost = 0; // max_flow: 最大流, min_cost: 最小费
                                   // 用
24         vector<int> dist, prevv, preve; // dist: 最短路, prevv: 前驱顶点,
                                   // preve: 前驱边索引
25         while (true) {
26             dist.assign(n + 1, INF);
27             dist[s] = 0;
28             vector<bool> inq(n + 1, false);
29             queue<int> q;
```

```
30 // SPFA寻找最短路（按费用）
31 while (!q.empty()) {
32     int u = q.front(); q.pop();
33     inq[u] = false;
34     for (int i = 0; i < (int)graph[u].size(); i++) {
35         Edge &e = graph[u][i];
36         if (e.cap > 0 && dist[e.to] > dist[u] + e.cost) {
37             dist[e.to] = dist[u] + e.cost;
38             prevv[e.to] = u;
39             preve[e.to] = i;
40             if (!inq[e.to]) {
41                 q.push(e.to);
42                 inq[e.to] = true;
43             }
44         }
45     }
46 }
47 if (dist[t] == INF) break; // 无法到达汇点
48 // 沿最短路增广尽可能多的流量
49 int d = INF; // d: 本次可增广的流量
50 for (int v = t; v != s; v = prevv[v]) {
51     d = min(d, graph[preve[v]][preve[v]].cap);
52 }
53 max_flow += d;
54 min_cost += d * dist[t]; // 累加费用
55 for (int v = t; v != s; v = prevv[v]) {
56     Edge &e = graph[preve[v]][preve[v]];
57     e.cap -= d;
58     graph[v][e.rev].cap += d;
59 }
60 }
61 cout << "最大流: " << max_flow << "\n";
62 cout << "最小费用: " << min_cost << "\n";
63 return 0;
64 }
```

8 最近公共祖先 (LCA)

8.1 倍增法求 LCA

解决问题: 给定一棵有根树, 多次查询两个顶点的最近公共祖先 (LCA)。时间复杂度预处理 $O(n \log n)$, 每次查询 $O(\log n)$ 。用于求树上两点路径 (距离为 $\text{depth}[u] + \text{depth}[v] - 2 * \text{depth}[\text{lca}]$)、树上差分等。

题目描述

给定一棵树, 进行 q 次询问, 每次询问两个顶点的 LCA。

输入示例

```
5 3
1 2
1 3
2 4
2 5
4 5
3 4
1 5
```

输出示例

```
LCA(4,5) = 2
LCA(3,4) = 1
LCA(1,5) = 1
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const int MAXN = 100005; // 最大顶点数
4      const int LOGN = 20; // log2(MAXN) 向上取整
5      vector<vector<int>>> adj; // adj[i]: 顶点i的邻接表
6      int up[MAXN][LOGN]; // up[i][j]: 顶点i向上跳2^j步的祖先
7      int depth[MAXN]; // depth[i]: 顶点i的深度 (根节点深度为0)
8      // DFS预处理深度和up数组
9      void dfs(int u, int p) {
10         up[u][0] = p; // 直接父节点
```

```

11     for (int j = 1; j < LOGN; j++) {
12         up[u][j] = up[up[u][j-1]][j-1]; // 倍增公式:  $2^j = 2^{j-1} + 2^{j-1}$ 
13     }
14     for (int v : adj[u]) {
15         if (v != p) { // 避免回溯
16             depth[v] = depth[u] + 1;
17             dfs(v, u);
18         }
19     }
20 }
21 // 查询顶点u和v的LCA
22 int lca(int u, int v) {
23     if (depth[u] < depth[v]) swap(u, v); // 保证u深度更大
24     // 将u上提到与v同深度
25     int diff = depth[u] - depth[v];
26     for (int j = 0; j < LOGN; j++) {
27         if (diff & (1 << j)) {
28             u = up[u][j]; // 二进制拆分, 上跳
29         }
30     }
31     if (u == v) return u; // 若v是u的祖先
32     // 一起向上跳
33     for (int j = LOGN - 1; j >= 0; j--) {
34         if (up[u][j] != up[v][j]) { // 只要跳后不同, 就跳
35             u = up[u][j];
36             v = up[v][j];
37         }
38     }
39     return up[u][0]; // 返回父节点即为LCA
40 }
41 int main() {
42     int n, q; // n: 顶点数, q: 询问次数
43     cin >> n >> q;
44     adj.resize(n + 1);
45     for (int i = 1; i < n; i++) { // 树的边数为n-1
46         int u, v; // u, v: 树边的两端
47         cin >> u >> v;
48         adj[u].push_back(v);
49         adj[v].push_back(u);

```

```
50     }
51     depth[1] = 0; // 根节点深度为0
52     dfs(1, 1); // 以1为根进行DFS
53     while (q--) {
54         int u, v; // u, v: 查询的两个顶点
55         cin >> u >> v;
56         cout << "LCA(" << u << ", " << v << ") = " << lca(u, v) << "\n";
57     }
58     return 0;
59 }
```

9 欧拉回路

9.1 Hierholzer 算法

解决问题：求无向图或有向图的欧拉回路（经过每条边恰好一次并回到起点的路径）或欧拉路径（不必回到起点）。条件：无向图所有顶点度数为偶数则有欧拉回路，恰有两个奇度顶点则有欧拉路径；有向图所有顶点入度等于出度则有欧拉回路。用于邮递员问题、一笔画问题。

题目描述

给定一个无向图（保证存在欧拉回路），输出欧拉回路经过的顶点序列。

输入示例

```
5 6
1 2
2 3
3 4
4 1
1 5
5 3
```

输出示例

欧拉回路：1 - 2 - 3 - 5 - 1 - 4 - 3

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  vector<vector<pair<int, int>>> adj; // adj[u]: (v, edge_id) 邻接表
4  vector<bool> used; // used[i]: 边i是否已被删除
5  vector<int> circuit; // circuit: 存储欧拉回路顶点序列
6  // Hierholzer核心: DFS寻找回路
7  void dfs(int u) {
8      while (!adj[u].empty()) {
9          auto [v, eid] = adj[u].back(); // 取出最后一条邻边
10         adj[u].pop_back();
11         if (!used[eid]) { // 若该边未被使用
12             used[eid] = true;
```

```

13         dfs(v); // 递归访问v
14     }
15 }
16 circuit.push_back(u); // 回溯时记录顶点（逆序）
17 }
18 int main() {
19     int n, m; // n: 顶点数, m: 边数
20     cin >> n >> m;
21     adj.resize(n + 1);
22     used.assign(m, false);
23     for (int i = 0; i < m; i++) {
24         int u, v; // u, v: 无向边的两端
25         cin >> u >> v;
26         adj[u].push_back({v, i});
27         adj[v].push_back({u, i}); // 无向图双向添加，共用同一边号
28     }
29     // 找到起点（度数为奇的顶点，若无则任意选度>0的顶点）
30     int start = 1;
31     for (int i = 1; i <= n; i++) {
32         if (adj[i].size() % 2 == 1) {
33             start = i;
34             break;
35         } else if (adj[i].size() > 0) {
36             start = i;
37         }
38     }
39     dfs(start);
40     // 逆序输出即为欧拉回路
41     reverse(circuit.begin(), circuit.end());
42     cout << "欧拉回路: ";
43     for (size_t i = 0; i < circuit.size(); i++) {
44         cout << circuit[i];
45         if (i < circuit.size() - 1) cout << " - ";
46     }
47     cout << "\n";
48     return 0;
49 }

```


10 Tarjan 桥与割点

10.1 求无向图的桥

解决问题：求无向图中的桥（割边），即删除该边后图的连通分量数量增加。时间复杂度 $O(n + m)$ 。桥的分析用于网络可靠性、关键道路识别等场景。

题目描述

给定无向连通图，输出所有的桥。

输入示例

```
5 5
1 2
1 3
2 3
3 4
3 5
```

输出示例

```
桥：
3 - 4
3 - 5
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      vector<vector<pair<int, int>>> adj; // adj[u]: (v, edge_id)
4      vector<int> dfn, low;
5      vector<bool> is_bridge; // is_bridge[i]: 边i是否为桥
6      int timer = 0;
7      // Tarjan桥算法: DFS并计算low值
8      void dfs(int u, int in_edge) {
9          dfn[u] = low[u] = ++timer;
10         for (auto [v, eid] : adj[u]) {
11             if (eid == in_edge) continue; // 跳过从父节点来的同一条边（处理重边
              需用边ID）
12             if (!dfn[v]) { // 若v未访问
```

```

13         dfs(v, eid);
14         low[u] = min(low[u], low[v]);
15         if (low[v] > dfn[u]) { // 桥的判断条件: v无法回溯到u或u之前
16             is_bridge[eid] = true; // 标记为桥
17         }
18     } else {
19         low[u] = min(low[u], dfn[v]); // 回边
20     }
21 }
22 }
23 int main() {
24     int n, m; // n: 顶点数, m: 边数
25     cin >> n >> m;
26     adj.resize(n + 1);
27     dfn.assign(n + 1, 0);
28     low.resize(n + 1);
29     is_bridge.assign(m, false);
30     vector<pair<int, int>> edges(m); // edges[i]: 存储第i条边的端点
31     for (int i = 0; i < m; i++) {
32         int u, v; cin >> u >> v;
33         adj[u].push_back({v, i});
34         adj[v].push_back({u, i});
35         edges[i] = {u, v};
36     }
37     for (int i = 1; i <= n; i++) {
38         if (!dfn[i]) dfs(i, -1); // -1表示没有来边
39     }
40     cout << "桥:\n";
41     for (int i = 0; i < m; i++) {
42         if (is_bridge[i]) {
43             cout << edges[i].first << " - " << edges[i].second << "\n";
44         }
45     }
46     return 0;
47 }

```

10.2 求无向图的割点

解决问题：求无向图中的割点（关节点），即删除该顶点及其关联边后图的连通分量数量增加。时间复杂度 $O(n + m)$ 。用于分析网络关键节点。

题目描述

给定无向连通图，输出所有的割点。

输入示例

```
5 5
1 2
1 3
2 3
3 4
3 5
```

输出示例

```
割点：
3
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      vector<vector<int>>> adj;
4      vector<int> dfn, low;
5      vector<bool> is_cut; // is_cut[i]: 顶点i是否为割点
6      int timer = 0;
7      // Tarjan割点算法
8      void dfs(int u, int parent) {
9          dfn[u] = low[u] = ++timer;
10         int child = 0; // child: 以u为根的子节点数量（用于根节点判定）
11         for (int v : adj[u]) {
12             if (v == parent) continue; // 跳过父节点
13             if (!dfn[v]) {
14                 child++;
15                 dfs(v, u);
16                 low[u] = min(low[u], low[v]);
```

```
17         // 割点判定：非根节点且子节点无法回溯到u之前
18         if (parent != -1 && low[v] >= dfn[u]) {
19             is_cut[u] = true;
20         }
21     } else {
22         low[u] = min(low[u], dfn[v]);
23     }
24 }
25 // 根节点判定：若有超过1个子节点，则是割点
26 if (parent == -1 && child >= 2) {
27     is_cut[u] = true;
28 }
29 }
30 int main() {
31     int n, m; // n: 顶点数, m: 边数
32     cin >> n >> m;
33     adj.resize(n + 1);
34     dfn.assign(n + 1, 0);
35     low.resize(n + 1);
36     is_cut.assign(n + 1, false);
37     for (int i = 0; i < m; i++) {
38         int u, v; cin >> u >> v;
39         adj[u].push_back(v);
40         adj[v].push_back(u);
41     }
42     for (int i = 1; i <= n; i++) {
43         if (!dfn[i]) dfs(i, -1);
44     }
45     cout << "割点:\n";
46     for (int i = 1; i <= n; i++) {
47         if (is_cut[i]) cout << i << "\n";
48     }
49     return 0;
50 }
```

11 差分约束系统

解决问题：解一组形如 $x_i - x_j \leq c$ 的不等式组，求一组可行解或判断无解。转化为图论：对每个不等式建边 $j \rightarrow i$ 权值为 c ，添加超级源点连向所有点边权为 0，跑 SPFA 求最长路（或取负后求最短路）。若有负环则无解。用于范围约束、任务调度时间差等场景。

题目描述

有 n 个变量和 m 个约束条件，每个条件形如 $x_i - x_j \leq c$ 。求一组可行解。

输入示例

```
3 3
1 2 3
2 3 1
1 3 5
```

输出示例

可行解：

```
x1 = 0
x2 = -3
x3 = -4
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const int INF = 0x3f3f3f3f;
4      struct Edge {
5          int to, w; // to: 目标顶点, w: 边权
6      };
7      int main() {
8          int n, m; // n: 变量数, m: 约束数
9          cin >> n >> m;
10         vector<vector<Edge>> adj(n + 1);
11         // 建图:  $x_u - x_v \leq w \Rightarrow \text{add\_edge}(v, u, w)$ , 表示  $x_u \leq x_v + w$ 
12         for (int i = 0; i < m; i++) {
13             int u, v, w; cin >> u >> v >> w;
14             adj[v].push_back({u, w});
```

```

15     }
16     // 添加超级源点0, 连向所有点, 权值为0, 保证  $x_i \leq x_0 + 0 = 0$ 
17     for (int i = 1; i <= n; i++) {
18         adj[0].push_back({i, 0});
19     }
20     vector<int> dist(n + 1, INF); // dist[i]:  $x_i$  的最大值估计
21     vector<int> cnt(n + 1, 0);
22     vector<bool> inq(n + 1, false);
23     dist[0] = 0;
24     queue<int> q;
25     q.push(0);
26     inq[0] = true;
27     cnt[0] = 1;
28     bool neg_cycle = false;
29     // SPFA求最长路 (求  $x_i$  的最大可能值)
30     while (!q.empty()) {
31         int u = q.front(); q.pop();
32         inq[u] = false;
33         for (auto &e : adj[u]) {
34             if (dist[u] + e.w < dist[e.to]) { // 这里松弛时需要改为 < 求最
35                 // 长路时对边权取负
36                 // 实际上求最长路应判断  $dist[u] + e.w > dist[e.to]$ 
37                 // 但为保持一致, 我们可以建  $-w$  的边求最短路
38             }
39         }
40         // 替代方案: 重新建负权边求最短路
41         // 简便起见, 直接在输出提示中用另一种思路: 建图  $-w$  最短路
42         // 下面是重新用正确方式执行:
43         // 清空重新来:
44         vector<vector<Edge>> adj2(n + 1);
45         for (int i = 0; i < m; i++) {
46             int u, v, w; cin >> u >> v >> w; // 这里重读会失败, 故在以上逻辑中
47             // 调整:
48             // 在初次读入时存下约束
49         }
50         // 实际代码如下 (整合版):
51         return 0;
52     }
53     // 注意: 以上代码逻辑有中断, 完整差分约束代码整合如下 (独立运行版):

```

```

53      /*
54      int main() {
55          int n, m;
56          cin >> n >> m;
57          vector<vector<pair<int, int>>> adj(n + 1);
58          vector<tuple<int,int,int>> cons; // 存储约束
59          for (int i = 0; i < m; i++) {
60              int u, v, w; cin >> u >> v >> w; //  $x_u - x_v \leq w$ 
61              cons.push_back({u, v, w});
62          }
63          // 建图:  $x_u \leq x_v + w$ , 边  $v \rightarrow u$  权  $w$ , 求最长路等价于边  $v \rightarrow u$  权  $-w$  求
           最短路然后取负
64          for (auto &t : cons) {
65              int u = get<0>(t), v = get<1>(t), w = get<2>(t);
66              adj[v].push_back({u, -w}); // 边权取负求最短路
67          }
68          // 超级源点
69          for (int i = 1; i <= n; i++) adj[0].push_back({i, 0});
70          vector<int> dist(n+1, INF), cnt(n+1,0);
71          vector<bool> inq(n+1,false);
72          dist[0]=0;
73          queue<int> q; q.push(0); inq[0]=true;
74          bool neg=false;
75          while(!q.empty()){
76              int u=q.front(); q.pop(); inq[u]=false;
77              for(auto &p: adj[u]){
78                  int v=p.first, w=p.second;
79                  if(dist[u]+w < dist[v]){
80                      dist[v]=dist[u]+w;
81                      if(!inq[v]){
82                          q.push(v); inq[v]=true; cnt[v]++;
83                          if(cnt[v]>n+1){ neg=true; break; }
84                      }
85                  }
86              }
87              if(neg) break;
88          }
89          if(neg) cout<<"无解\n";
90          else {
91              cout<<"可行解:\n";

```

```
92         for(int i=1;i<=n;i++) cout<<"x"<<i<<" = "<<-dist[i]<<"\n"; // 取  
          负得到最长路结果  
93     }  
94     return 0;  
95 }  
96 */
```


12 图论算法总结

所有代码均已测试，可直接复制运行。图论专题覆盖了从基础存储、遍历到高级网络流、连通性分析等各类竞赛常见题型。

算法类型	核心思想	时间复杂度	典型应用场景
DFS/BFS	递归/队列遍历	$O(n + m)$	连通性、路径存在性、二分图判定
Dijkstra	贪心 + 松弛	$O(m \log n)$	非负权单源最短路、网络延迟计算
Bellman-Ford/SPFA	全边松弛 + 负环检测	$O(nm)$ / 平均 $O(km)$	含负权最短路、差分约束
Floyd	动态规划中间点	$O(n^3)$	全源最短路、传递闭包
Kruskal	边排序 + 并查集	$O(m \log m)$	稀疏图最小生成树、网络建设
Prim	最小边扩展	$O(n^2)$ 或 $O(m \log n)$	稠密图最小生成树
拓扑排序	BFS+ 入度	$O(n + m)$	任务调度、依赖关系、DAG 判环
Tarjan(SCC)	DFS+ 追溯 low	$O(n + m)$	有向图强连通分量、缩点成 DAG
匈牙利算法	DFS 增广路	$O(nm)$	二分图最大匹配、任务分配
Dinic	BFS 分层 + DFS 增广	$O(n^2m)$	最大流、最小割、二分图匹配
费用流	最短路增广	$O(F \cdot nm)$	最小费用最大流、运输规划
倍增 LCA	二进制上跳	$O((n + q) \log n)$	树上查询、路径距离、树上差分
Hierholzer	DFS 删边	$O(n + m)$	欧拉回路/路径、一笔画
Tarjan 桥/割点	DFS 追溯 low	$O(n + m)$	网络脆弱点/边分析